Eigenvalues in Plankton Blooms

Jacob Bonner, August Hauter, Ryan Sabin

APPM 2360

April 2019

1 Introduction

This report analyzes two methods for the computation of eigenvalues in large systems: the QR Algorithm and the Power Method. To show how these methods are used, we consider a model describing the size of plankton populations, and derive a condition based on the smallest eigenvalue of the system for the population to grow. Phytoplankton provide 50 % of the world's oxygen and consume carbon dioxide on a scale equivalent to forests and other land plants, therefore it is important to know when a plankton bloom will be growing or decaying.

We begin our analysis by modeling the population of Phytoplankton using differential equations, and solving for solutions to explore the behavior of population growth. By defining how the population changes in space and time, we develop solutions that allow us to find the minimum conditions for plankton growth.

Following our investigation of population models, we discretize an approximation for the second derivative to analyze the eigenvalues of our system. We derive a matrix-vector equation that forms an approximation for our eigenvalue problem, and is later used in QR Algorithm and the power method.

For computing the eigenvalues of our plankton system, we first consider a simple strategy for finding the eigenvalues of an $n \times n$ Matrix $\mathbf{A}$. Using the equation $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$, which can be written as $(\mathbf{A} - \lambda\mathbf{I})\mathbf{v} = 0$, we know that nonzero solutions exist only if the coefficient matrix is singular. Therefore, to find the eigenvalues of $\mathbf{A}$, we must have $|\mathbf{A} - \lambda\mathbf{I}| = 0$.

With that in mind, our first method for large systems is the QR Algorithm, which takes the original Matrix $\mathbf{A}$ and finds a sequence of matrices similar to $\mathbf{A}$ that converge to an upper triangular matrix. Once we have the upper triangular matrix $\mathbf{R}$ that is similar to $\mathbf{A}$, meaning they have the same eigenvalues, then we are able to simply extract the eigenvalues from the diagonal of $\mathbf{R}$.

The power method is different in the sense that we use it to compute the extreme eigenvalues, in other words, the largest and smallest eigenvalues of a matrix $\mathbf{A}$. First, we assume that the matrix $\mathbf{A}$ has a maximum eigenvalue with corresponding eigenvectors. Then, through many iterations we obtain an approximation for the largest eigenvalue of our matrix. However, in this case we are looking for the smallest eigenvalue, so we use the power method on $\mathbf{A}^{-1}$ and take the reciprocal of the resultant value.

2 Modeling Plankton Population

The simplest growth model for the population of Phytoplankton is the equation

$$\frac{dN}{dt} = (K - \lambda)N$$

where N is the population, K is the growth rate, and λN is the rate of population loss by diffusion. Here we see that the population will experience growth if $K > \lambda$. However, since the population is modeled with diffusion, we instead use the diffusion term in the simple growth model, obtaining the partial differential equation

$$\frac{\partial c}{\partial t} = D\frac{\partial^2 c}{\partial x^2} + Kc \tag{1}$$

where $c(x,t)$ is the concentration of plankton, describing growth in both space and time. Here, the diffusion term is summed with the term Kc, which represents the linear growth of the plankton bloom. We assume the concentration is zero at the ends of an interval of length l, giving the boundary conditions $u(0,t) = u(l,t) = 0$.

We then make the transformation

$$c(x,t) = u(x,t)e^{Kt} \tag{2}$$

and by substituting (2) into (1), we reduce to the diffusion equation (Appendix 1)

$$\frac{\partial u}{\partial t} = D\frac{\partial^2 u}{\partial x^2} \tag{3}$$

3 The Exact Solution

To investigate the behavior of the plankton population we solve equation (3) using separation of variables when the diffusion coefficient D is constant. We assume a solution of the form $u(x,t) = X(x)T(t)$, and obtain the following two equations by substituting into equation (3).

$$T' + \lambda DT = 0 \tag{4}$$

$$X'' + \lambda X = 0 \tag{5}$$

Here, λ represents the eigenvalues and the solution to the diffusion equation.

The boundary conditions are $X(0) = X(l) = 0$, which by way of separation of variables leads to the eigenvalues λ_n, and eigenfunctions (Appendix 1)

$$X_n(x) = sin(\sqrt{\lambda_n}x)$$
$$T_n(t) = e^{-D\lambda_n t}$$

where each product $X_n(x)T_n(t)$ is a solution of (3), and satisfies the boundary conditions.

We then obtain the general solution of equation (3) by taking the weighted sum of all solutions of the form $X_n(x)T_n(t)$:

$$u(x,t) = \sum_{n=1}^{\infty} A_n sin(\sqrt{\lambda_n}x)e^{-D\lambda_n t} \tag{6}$$

This weighted sum of solutions is also a solution to the differential equation because (3) is a linear operator (Appendix 1). Thus, the superposition principle for linear homogeneous equations applies, meaning that the sum of solutions to $L(u) = 0$ are also solutions.

By substituting (6) into equation (2), the concentration of plankton becomes (Appendix 1)

$$c(x,t) = \sum_{n=1}^{\infty} A_n sin(\sqrt{\lambda_n} x) e^{(K - D\lambda_n)t} \tag{7}$$

where the constants A_n are determined by the initial condition $c(x, 0)$.

The plankton population will grow when the growth rate $K > \lambda_1$, where λ_1 is the smallest eigenvalue, known as

$$\lambda_1 = \left(\frac{\pi}{l}\right)^2 \tag{8}$$

If the growth rate $K < D\lambda_n$ for all n, the plankton population will exponentially decay. However, if $K > D\lambda_n$ for at least one n, the plankton bloom will experience growth. Therefore, when $K > D\lambda_1$, the concentration $c(x,t)$ will experience exponential growth because the exponent of $e^{(K-D\lambda_n)t}$ will be positive. In this case, K being greater than $D\lambda_1$ represents the minimum condition for growth, as the population would also grow when $K > D\lambda_n$ for other values of n.

4 Discretization

We now show how discretization of the eigenvalue problem leads to the smallest eigenvalue. To do this we discretize the operator X'' to obtain a matrix eigenvalue problem that approximates equation (5). We find an approximation for the second derivative, X'', by applying two definitions of the first derivative to X.

The derivative of a single valued function f at a point x can be defined as the forward difference approximation

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \approx \frac{f(x+h) - f(x)}{h} \tag{9}$$

or the backward difference approximation

$$f'(x) = \lim_{h \to 0} \frac{f(x) - f(x-h)}{h} \approx \frac{f(x) - f(x-h)}{h} \tag{10}$$

We derive the approximation for X'' by applying (9) to X, resulting in an approximation for X', and then applying (10) to our approximation for X' (Appendix 1). This process results in the following approximation for the second derivative.

$$X''(x) \approx \frac{X(x-h) - 2X(x) + X(x+h)}{h^2} \tag{11}$$

Now, we discretize the interval $(0, l)$ into $n+1$ parts of length $h = l/(n+1)$. Letting $x_k = X(hk)$, we derive a system of equations to approximate (5) by writing out the equations for $-X'' = \lambda X$ at the points x_1, x_2, x_3, and so on (Appendix 1). The eigenvalue problem (5) is then approximated by the matrix-vector eigenvalue problem

$$\frac{1}{h^2} \begin{bmatrix} 2 & -1 & & & & & \\ -1 & 2 & -1 & & & & \\ & -1 & 2 & -1 & & & \\ & & \ddots & \ddots & \ddots & & \\ & & & \ddots & \ddots & \ddots & \\ & & & & \ddots & 2 & -1 \\ & & & & & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$$

This equation represents the interior points, as we have the boundary conditions $X(0) = X(l) = 0$, corresponding to $x_0 = x_{n+1} = 0$.

Letting B_n be the $n \times n$ matrix above, the eigenvalue problem can be written as

$$\left(\frac{n+1}{l}\right)^2 B_n \mathbf{x} = \lambda \mathbf{x}$$

We now define the matrix $A_n \equiv \left(\frac{n+1}{l}\right)^2 B_n$. For simplicity we assume that $l = 1$, so A reduces to $A_n = (n+1)^2 B_n$, and equation (8) becomes $\lambda_1 = \pi^2$. The eigenvalues of this matrix A_n approximate the eigenvalues of the plankton model. As $n \to \infty$ the approximations become increasingly accurate, which we explore in the following sections.

5 QR Algorithm

After creating the matrix A_n, we now use the QR Algorithm to find the eigenvalues of A for different values of n. To better understand how this algorithm works we first examine the Gram-Schmidt Algorithm and QR Factorization.

Gram-Schmidt Algorithm

Given a set of linearly independent vectors $\{\mathbf{v}_i\}_{i=1}^n$ it is possible to create an orthonormal basis $\{\mathbf{u}_i\}_{i=1}^n$, where each vector $\mathbf{u}_n$ has magnitude 1, and is orthogonal to every other vector in the set. To illustrate this process, we analyze low-dimensional cases and build up an algorithm for any given number of vectors.

If we have a single vector in our set $\{u\}$, we construct an orthonormal basis by setting

$$\mathbf{u} = \frac{\mathbf{v}}{\|\mathbf{v}\|}$$

When we have two vectors in the set $\{\mathbf{v}_1, \mathbf{v}_2\}$, we start by creating the vector

$$\mathbf{u}_1 = \frac{\mathbf{v}_1}{\|\mathbf{v}_1\|}$$

and then find an additional vector that is orthogonal to $\mathbf{v}_1$. To obtain this new vector we write the vector $\mathbf{v}_2$ in terms of its components parallel and perpendicular to the line spanned by $\mathbf{u}_1$.

$$\mathbf{v}_2 = \mathbf{v}_2^{\parallel} + \mathbf{v}_2^{\perp}$$

Now we have the vector $\mathbf{v}_2^{\perp}$, which is orthogonal to $\mathbf{u}_1$, and can be written as

$$\mathbf{v}_2^{\perp} = \mathbf{v}_2 - \mathrm{proj}_{\mathrm{span}\{\mathbf{u}_1\}}(\mathbf{v}_2) = \mathbf{v}_2 - (\mathbf{u}_1 \cdot \mathbf{v}_2)\mathbf{u}_1$$

To find the second vector $\mathbf{u}_2$ of the orthonormal basis we divide $\mathbf{v}_2^{\perp}$ by its length, obtaining

$$\mathbf{u}_2 = \frac{\mathbf{v}_2^{\perp}}{\|\mathbf{v}_2^{\perp}\|}$$

To generalize this process for a given number of n vectors, we use the following algorithm.

$$\text{Set } \mathbf{u}_1 = \mathbf{v}_1$$

$$\mathbf{u}_n = \sum_{k=1}^{n-1} \frac{(\mathbf{u}_k, \mathbf{v}_n)}{\|\mathbf{u}_k\|^2} \mathbf{u}_k$$

We start by setting the first orthonormal basis vector $\mathbf{u}_1$ equal to the first vector in the set $\{\mathbf{v}_i\}_{i=1}^n$. Then to obtain the remaining vectors, we subtract off the projection of $\mathbf{v}_n$ onto the subspace spanned by $\{\mathbf{u}_i\}_{i=1}^n$ to find a vector orthogonal to the previous vectors. The final step is to divide each vector by its magnitude in order to create the orthonormal basis.

We now find an orthonormal basis for the column space of the 3×3 matrix A_3. To verify that the vectors $\mathbf{u}_1, \mathbf{u}_2, \mathbf{u}_3$ do in fact form an orthonormal basis, we show that they are orthogonal, that is, that the dot product $(\mathbf{u}_i, \mathbf{u}_j) = 0$, or numerical 0, for $i \neq j$. Also, we prove that each vector has length 1. (Appendix 2)

$$\mathbf{u}_1 = \begin{bmatrix} 0.8944 \\ -0.4472 \\ 0 \end{bmatrix} \quad \mathbf{u}_2 = \begin{bmatrix} 0.3586 \\ 0.7171 \\ -0.5976 \end{bmatrix} \quad \mathbf{u}_3 = \begin{bmatrix} 0.2673 \\ 0.5345 \\ 0.8018 \end{bmatrix}$$

QR Factorization

The Gram-Schmidt Algorithm takes in an "old" basis and produces a "new" orthonormal basis. This process can be described by a change of basis matrix $\mathbf{R}$. We write this transformation with the following equation.

$$\begin{aligned} \begin{bmatrix} \mathbf{v}_1 & \cdots & \cdots & \mathbf{v}_n \end{bmatrix} &= \begin{bmatrix} \mathbf{u}_1 & \cdots & \cdots & \mathbf{u}_n \end{bmatrix} \mathbf{R} \\ &= \mathbf{QR} \end{aligned}$$

where $\mathbf{Q}$ is an orthogonal matrix through construction, and $\mathbf{R}$ is an upper triangular matrix.

We now obtain the QR factorization for A_n matrices of size 3×3 and 4×4. (Appendix 2)

$$Q_3 = \begin{bmatrix} 0.8944 & 0.3586 & 0.2673 \\ -0.4472 & 0.7171 & 0.5345 \\ 0 & -0.5976 & 0.8018 \end{bmatrix} \quad R_3 = \begin{bmatrix} 35.7771 & -28.6217 & 7.1554 \\ 0 & 26.7731 & -30.5979 \\ 0 & 0 & 17.1047 \end{bmatrix}$$

$$Q_4 = \begin{bmatrix} 0.8944 & 0.3586 & 0.1952 & 0.1826 \\ -0.4472 & 0.7171 & 0.3904 & 0.3651 \\ 0 & -0.5976 & 0.5855 & 0.5477 \\ 0 & 0 & -0.6831 & 0.7303 \end{bmatrix} \quad R_4 = \begin{bmatrix} 55.9017 & -44.7214 & 11.1803 & 0 \\ 0 & 41.8330 & -47.8091 & 14.9404 \\ 0 & 0 & 36.5963 & -48.7950 \\ 0 & 0 & 0 & 22.8218 \end{bmatrix}$$

QR Algorithm in Practice

Upon examining Gram-Schmidt and QR Factorization, we now utilize the QR Algorithm in its entirety. The following steps of the QR algorithm are used to compute the eigenvalues of a given matrix $\mathbf{A}$.

$$\begin{aligned} &\text{Step 0: Set } \mathbf{A}_0 = \mathbf{A} \\ &\text{Step k: Set } \mathbf{A}_k = \mathbf{Q}_k\mathbf{R}_k \text{ and then set } \mathbf{A}_{k+1} = \mathbf{R}_k\mathbf{Q}_k \end{aligned}$$

Here the matrix $\mathbf{A}_{k+1}$ is similar to the matrix $\mathbf{A}_k$, meaning that the matrices have the same eigenvalues. We verify this fact by showing that the characteristic equations $|A_k - \lambda I|$ and $|A_{k+1} - \lambda I|$ are equivalent (Appendix 1). Under certain conditions $\mathbf{A}_k$ converges to an upper triangular matrix, or a diagonal matrix if $\mathbf{A}_k$ is symmetric.

We now apply the QR Algorithm 1000 times to $\mathbf{A}_n$ of sizes 10×10, 20×20, and 100×100, which will result in a diagonal matrix. Then we extract the smallest eigenvalue from the diagonal of each matrix. These values are given below. (Appendix 2)

$$\begin{aligned} \text{Smallest eigenvalue for } A_{10}: &\quad \lambda = 9.8027 \\ \text{Smallest eigenvalue for } A_{20}: &\quad \lambda = 9.8512 \\ \text{Smallest eigenvalue for } A_{100}: &\quad \lambda = 9.8688 \end{aligned}$$

Next, we compare the eigenvalue of each A_n to our known smallest eigenvalue $\lambda_1 = \pi^2$. To do this we calculate the percent error between the smallest λ and λ_1 for each matrix. (Appendix 2)

$$\begin{aligned} \text{Percent Error for } A_{10}: &\quad \lambda = 0.6779\,\% \\ \text{Percent Error for } A_{20}: &\quad \lambda = 0.1864\,\% \\ \text{Percent Error for } A_{100}: &\quad \lambda = 0.0081\,\% \end{aligned}$$

We see that the error of the QR Algorithm decreases as the size of the matrix A_n increases. In other words, as $n \to \infty$ the approximation from the QR Algorithm becomes progressively more accurate.

6 Power Method

In this section we analyze the extreme eigenvalues of our system using the Power Method. We first give a description of the power method, and explain the process of solving for the largest eigenvalue of a system.

If we assume that there is an eigenbasis for our problem, we can write any vector as

$$\mathbf{x} = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n \quad c_i \in \mathbb{R}$$

Applying $\mathbf{A}$ to this equation we obtain

$$\mathbf{A}\mathbf{x} = c_1\mathbf{A}\mathbf{v}_1 + c_2\mathbf{A}\mathbf{v}_2 + \cdots + c_n\mathbf{A}\mathbf{v}_n \quad c_i \in \mathbb{R}$$

and using the eigenvalue relationship $\mathbf{A}\mathbf{v}_i = \lambda_i\mathbf{v}_i$ we arrive at the equation

$$\mathbf{A}\mathbf{x} = c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2 + \cdots + c_n\lambda_n\mathbf{v}_n \quad c_i \in \mathbb{R}$$

If we assume that the largest eigenvalue λ_1 is distinct from the others, in other words, $|\lambda_1| > |\lambda_2| \geq \cdots \geq |\lambda_n|$, we can factor out λ_1 and obtain the following equation.

$$\mathbf{A}\mathbf{x} = \lambda_1 \Big(c_1 \mathbf{v}_1 + c_2 \frac{\lambda_2}{\lambda_1} \mathbf{v}_2 + \cdots + c_n \frac{\lambda_n}{\lambda_1} \mathbf{v}_n \Big)$$

Repeated applications of $\mathbf{A}$ results in the equation

$$\mathbf{A}^n \mathbf{x} = \lambda_1^n \Big(c_1 \mathbf{v}_1 + c_2 \Big(\frac{\lambda_2}{\lambda_1} \Big)^n \mathbf{v}_2 + \cdots + c_n \Big(\frac{\lambda_n}{\lambda_1} \Big)^n \mathbf{v}_n \Big)$$

where $\left(\frac{\lambda_j}{\lambda_1}\right)^n$ will approach zero as $n \to \infty$ for $j \neq 1$.

Using the power method, we find the smallest eigenvalue of $\mathbf{A}_n$ for sizes 10×10, 20×20, and 100×100. For each $\mathbf{A}_n$, we first consider the following property of matrices regarding the eigenvalues of a matrix A and A^{-1}.

If λ is an eigenvalue of an invertible matrix $\mathbf{A}$, then $\dfrac{1}{\lambda}$ is an eigenvalue of $\mathbf{A}^{-1}$.

Thus, to extract the smallest eigenvalue of $\mathbf{A}_n$, we use the power method on $\mathbf{A}_n^{-1}$. The resultant value of λ from this process will be the largest eigenvalue of $\mathbf{A}_n^{-1}$. Then, our final step is to take the inverse of the resulting λ, as the largest eigenvalue of $\mathbf{A}_n^{-1}$ is the reciprocal of the smallest eigenvalue of $\mathbf{A}_n$. (Appendix 2)

After computation, the smallest eigenvalues we extract are as follows.

$$\begin{aligned} \text{Smallest eigenvalue for } A_{10}\text{:} \quad & \lambda = 9.8027 \\ \text{Smallest eigenvalue for } A_{20}\text{:} \quad & \lambda = 9.8512 \\ \text{Smallest eigenvalue for } A_{100}\text{:} \quad & \lambda = 9.8688 \end{aligned}$$

As we did for the QR Algorithm, we compare the resultant eigenvalue of each $\mathbf{A}_n$ to our known smallest eigenvalue, $\lambda_1 = \pi^2$. To do this we calculate the percent error between the smallest eigenvalue and λ_1 for each matrix. (Appendix 2)

$$\begin{aligned} \text{Percent Error for } A_{10}\text{:} \quad & \lambda = 0.6779\,\% \\ \text{Percent Error for } A_{20}\text{:} \quad & \lambda = 0.1864\,\% \\ \text{Percent Error for } A_{100}\text{:} \quad & \lambda = 0.0081\,\% \end{aligned}$$

Similar to the error of the previous method, the approximation of minimum eigenvalues gets increasingly accurate as the value of n (the size of the matrix) increases.

7 Comparison

After using both methods to compute the eigenvalues of our plankton population, we have developed a unanimous opinion on which method we prefer. We favor the power method because the process of finding the minimum eigenvalue made more sense, and it was easier to understand the mathematical process. Essentially, we were able to translate the instructions into actual code much more easily with the power method, as well as clearly find the minimum eigenvalue of $\mathbf{A}$ through the maximum eigenvalue of $\mathbf{A}^{-1}$. Additionally, despite the higher number of iterations in the power method, we only required one function to carry out the computations, while the entire QR method needed two. In general, we feel that our efficiency in using the the power method was higher, due to a better overall understanding, and a comparatively less amount of code. For these reasons, we prefer the power method in computing eigenvalues of large systems.

8 Conclusion

Upon analyzing the two eigenvalue computation methods, QR Algorithm and Power Method, we conclude that the two are equally accurate, and that the approximations grow closer to the actual value as the size of our matrix $\mathbf{A}_n$ increases. Not only do the methods compute identical eigenvalues, the error between the approximation and the actual value is relatively small. Therefore, either method can be utilized to find the minimum eigenvalues of the growth model of Phytoplankton in our world's oceans, with the same level of exceptional accuracy. This precision is important because of the vital role that plankton play in the environment, and how it provides the ability to verify any findings that concern the minimum conditions for plankton growth.

Appendix 1

Calculations for Section 2

Substitution of (2) into (1) that results in (3)

$$\text{Given} \quad c(x,t) = u(x,t)e^{Kt}$$

$$\frac{\partial c}{\partial t} = \frac{\partial u}{\partial t}e^{Kt} + u(x,t)Ke^{Kt} \quad \text{and} \quad \frac{\partial c}{\partial x} = \frac{\partial u}{\partial x}e^{Kt} \implies \frac{\partial^2 c}{\partial x^2} = \frac{\partial^2 u}{\partial x^2}e^{Kt}$$

$$\text{Substitute values into } \tfrac{\partial c}{\partial t} = D\tfrac{\partial^2 c}{\partial x^2} + Kc$$

$$\begin{aligned}
\frac{\partial u}{\partial t}e^{Kt} + u(x,t)Ke^{Kt} &= D(\frac{\partial^2 u}{\partial x^2}e^{Kt}) + K(u(x,t)e^{Kt}) \\
\frac{\partial u}{\partial t}e^{Kt} &= D\frac{\partial^2 u}{\partial x^2}e^{Kt} + u(x,t)Ke^{Kt} - u(x,t)Ke^{Kt} \\
\frac{\partial u}{\partial t}e^{Kt} &= D\frac{\partial^2 u}{\partial x^2}e^{Kt} \\
\frac{\partial u}{\partial t} &= D\frac{\partial^2 u}{\partial x^2}
\end{aligned}$$

Calculations for Section 3

Solving (4) and (5) Using Separation of Variables

$$
\begin{aligned}
T' + \lambda DT &= 0 \\
\frac{dT}{dt} &= -\lambda DT \\
\frac{1}{T}dT &= -\lambda D dt \\
\int \frac{1}{T}dT &= \int -\lambda D dt \\
\ln \mid T \mid &= -\lambda D t \\
T(t) &= e^{-\lambda D t} \\
T(t) &= e^{-D\lambda t} \\
T_n(t) &= e^{-D\lambda_n t}
\end{aligned}
$$

$$
\begin{aligned}
X'' + \lambda X &= 0 \\
\text{Try} \quad X(x) &= e^{rx} \\
r^2 e^{rx} + \lambda e^{rx} &= 0 \\
e^{rx}(r^2 + \lambda) &= 0 \\
r^2 + \lambda &= 0 \\
r &= \pm i\sqrt{\lambda} \\
X(x) &= c_1 e^{i\sqrt{\lambda}x} + c_2 e^{-i\sqrt{\lambda}x}
\end{aligned}
$$

Using Euler's Formula, $e^{i\theta} = cos(\theta) + isin(\theta)$, $X(x)$ becomes...

$$
\begin{aligned}
X(x) &= e^{(0)x}\big(c_1 cos(\sqrt{\lambda}x) + c_2 sin(\sqrt{\lambda}x)\big) \\
X(x) &= c_1 cos(\sqrt{\lambda}x) + c_2 sin(\sqrt{\lambda}x) \\
X(0) &= c_1 cos(\sqrt{\lambda}(0)) + c_2 sin(\sqrt{\lambda}(0)) \\
0 &= c_1 \\
X(x) &= c_2 sin(\sqrt{\lambda}x) \quad (\text{take } c_2 = 1) \\
X_n(x) &= sin(\sqrt{\lambda_n}x)
\end{aligned}
$$

Proving that (3) is a Linear Operator

$$\begin{aligned}
\frac{\partial u}{\partial t} &= D\frac{\partial^2 u}{\partial x^2} \\
\frac{\partial u}{\partial t} - D\frac{\partial^2 u}{\partial x^2} &= 0 \\
\Big(\frac{\partial}{\partial t} - D\frac{\partial^2}{\partial x^2}\Big)u &= 0
\end{aligned}$$

Define the operator $L = \dfrac{\partial}{\partial t} - D\dfrac{\partial^2}{\partial x^2}$. For linearity, L must satisfy these two properties:

$$\begin{aligned}
L(k\mathbf{u}) &= kL(\mathbf{u}), \quad k \in \mathbb{R} \\
L(\mathbf{u}+\mathbf{w}) &= L(\mathbf{u}) + L(\mathbf{w})
\end{aligned}$$

If u_1 and u_2 are solutions of $L(u) = 0$, and c_1,$c_2 \in \mathbb{R}$:

$$\begin{aligned}
L(c_1u_1 + c_2u_2) &= \Big(\frac{\partial}{\partial t} - D\frac{\partial^2}{\partial x^2}\Big)(c_1u_1 + c_2u_2) \\
&= \frac{\partial}{\partial t}(c_1u_1 + c_2u_2) - D\frac{\partial^2}{\partial x^2}(c_1u_1 + c_2u_2) \\
&= \frac{\partial}{\partial t}(c_1u_1) + \frac{\partial}{\partial t}(c_2u_2) - D\frac{\partial^2}{\partial x^2}(c_1u_1) - D\frac{\partial^2}{\partial x^2}(c_2u_2) \\
&= \frac{\partial}{\partial t}(c_1u_1) - D\frac{\partial^2}{\partial x^2}(c_1u_1) + \frac{\partial}{\partial t}(c_2u_2) - D\frac{\partial^2}{\partial x^2}(c_2u_2) \\
&= \Big(\frac{\partial}{\partial t} - D\frac{\partial^2}{\partial x^2}\Big)c_1u_1 + \Big(\frac{\partial}{\partial t} - D\frac{\partial^2}{\partial x^2}\Big)c_2u_2 \\
&= c_1L(u_1) + c_2L(u_2)
\end{aligned}$$

Using (2) to Derive the Solution for the Concentration (6)

$$\textbf{Substitute} \quad u(x,t) = \sum_{n=1}^{\infty} A_n sin(\sqrt{\lambda_n}x)e^{-D\lambda_n t} \quad \textbf{into} \quad c(x,t) = u(x,t)e^{Kt}$$

$$\begin{aligned}
c(x,t) &= \Big(\sum_{n=1}^{\infty} A_n sin(\sqrt{\lambda_n}x)e^{-D\lambda_n t}\Big)e^{Kt} \\
c(x,t) &= \sum_{n=1}^{\infty} A_n sin(\sqrt{\lambda_n}x)e^{(K-D\lambda_n)t}
\end{aligned}$$

Calculations for Section 4

Deriving the Approximation of the Second Derivative

$$X'(x) = \lim_{h\to 0} \frac{X(x+h) - X(x)}{h} \approx \frac{X(x+h) - X(x)}{h}$$

$$\begin{aligned}
X''(x) &= \lim_{h\to 0} \frac{X'(x) - X'(x-h)}{h} \\
&= \lim_{h\to 0} \frac{1}{h}\left(\frac{X(x+h) - X(x)}{h} - \frac{X((x-h)+h) - X(x-h)}{h}\right) \\
&= \lim_{h\to 0} \frac{1}{h}\left(\frac{X(x+h) - X(x) - X(x) + X(x-h)}{h}\right) \\
&= \lim_{h\to 0} \frac{X(x+h) - 2X(x) + X(x-h)}{h^2} \\
&\approx \frac{X(x-h) - 2X(x) + X(x+h)}{h^2}
\end{aligned}$$

Deriving the System of Equations that leads to the Matrix Equation

$$x_k = X(hk)$$

$$\begin{aligned}
X(hk+h) = X(h(k+1)) &= x_{k+1} \\
X(hk-h) = X(h(k-1)) &= x_{k-1} \\
x_k - h &= x_{k-1}
\end{aligned}$$

$$\begin{aligned}
-X'' &= \lambda X \\
-X''(x) &= \lambda X(x) \\
-\frac{X(x-h) - 2X(x) + X(x+h)}{h^2} &= \lambda X(x) \\
-\frac{x_{k-1} - 2x_k + x_{k+1}}{h^2} &= \lambda x_k \\
\frac{1}{h^2}\big(-x_{k-1} + 2x_k - x_{k+1}\big) &= \lambda x_k
\end{aligned}$$

Then we plug in different values of k

$$\begin{aligned}
(k=1) \qquad & \frac{1}{h^2}\big(-x_{1-1} + 2x_1 - x_{1+1}\big) = \lambda x_1 \implies \frac{1}{h^2}\big(2x_1 - x_2\big) = \lambda x_1 \\
(k=2) \qquad & \frac{1}{h^2}\big(-x_{2-1} + 2x_2 - x_{2+1}\big) = \lambda x_2 \implies \frac{1}{h^2}\big(-x_1 + 2x_2 - x_3\big) = \lambda x_2 \\
(k=3) \qquad & \frac{1}{h^2}\big(-x_{3-1} + 2x_3 - x_{3+1}\big) = \lambda x_3 \implies \frac{1}{h^2}\big(-x_2 + 2x_3 - x_4\big) = \lambda x_3 \\
(k=n-1) \qquad & \frac{1}{h^2}\big(-x_{n-1-1} + 2x_{n-1} - x_{n-1+1}\big) = \lambda x_{n-1} \implies \frac{1}{h^2}\big(-x_{n-2} + 2x_{n-1} - x_n\big) = \lambda x_{n-1} \\
(k=n) \qquad & \frac{1}{h^2}\big(-x_{n-1} + 2x_n - x_{n+1}\big) = \lambda x_n \implies \frac{1}{h^2}\big(-x_{n-1} + 2x_n\big) = \lambda x_n
\end{aligned}$$

Now we have the system of equations

$$\frac{1}{h^2}\big(2x_1 - x_2\big)$$

$$\frac{1}{h^2}\big(-x_1 + 2x_2 - x_3\big)$$

$$\frac{1}{h^2}\big(-x_2 + 2x_3 - x_4\big)$$

$$\frac{1}{h^2}\big(-x_{n-2} + 2x_{n-1} - x_n\big)$$

$$\frac{1}{h^2}\big(-x_{n-1} + 2x_n\big)$$

Which can be written as the following matrix

$$\frac{1}{h^2}\begin{bmatrix} 2 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 & 0 & 0 \\ 0 & 0 & \ddots & \ddots & \ddots & 0 & 0 \\ 0 & 0 & 0 & \ddots & \ddots & \ddots & 0 \\ 0 & 0 & 0 & 0 & \ddots & 2 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix} = \lambda \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ \vdots \\ \vdots \\ x_{n-1} \\ x_n \end{bmatrix}$$

Calculations for Section 5

Showing that the matrix A_{k+1} is similar to A_k

$$\begin{aligned} \text{Given} \quad A_k = Q_k R_k \quad &\implies \quad Q_k^{-1} A_k = R_k \\ A_{k+1} = R_k Q_k \quad &\implies \quad A_{k+1} = Q_k^{-1} A_k Q_k \end{aligned}$$

We want to show that $\quad |A_k - \lambda I| = |A_{k+1} - \lambda I|$

$$\begin{aligned} |A_{k+1} - \lambda I| &= |Q_k^{-1} A_k Q_k - \lambda I| \\ &= |Q_k^{-1} A_k Q_k - Q_k^{-1} \lambda Q_k| \\ &= |Q_k^{-1}(A_k - \lambda I) Q_k| \\ &= |Q_k^{-1}||A_k - \lambda I||Q_k| \\ &= \frac{1}{|Q_k|}|Q_k||A_k - \lambda I| \\ &= \frac{|Q_k|}{|Q_k|}|A_k - \lambda I| \\ &= |A_k - \lambda I| \end{aligned}$$

Appendix 2

MATLAB Functions

Function 1 - Creating Matrix A_n

```
function [A] = A(n)
    twos = diag(2*(ones(1,n)));
    neg1 = diag(-1*(ones(1,n-1)),1);
    neg2 = diag(-1*(ones(1,n-1)),-1);
    B = neg1 + neg2 + twos;
    A = (n+1)^2.*B;
end
```

Function 2 - Gram-Schmidt

```
function [u1, u2, u3] = gram(A)
    v1 = A(:,1);
    v2 = A(:,2);
    v3 = A(:,3);

    u1p = v1;
    u2p = v2 - (dot(u1p, v2)/((norm(u1p))^2))*u1p;
    u3p = v3 - ((dot(u1p, v3)/((norm(u1p))^2))*u1p + (dot(u2p, v3)/((norm(u2p))^2))*u2p);

    u1 = u1p/norm(u1p);
    u2 = u2p/norm(u2p);
    u3 = u3p/norm(u3p);
end
```

Function 3 - Applying the QR Algorithm

```
function [M] = QR(Ak)
    [Q R] = cgs(Ak);
    Ak = Q*R;
      for n = 1:999
          [Q R] = cgs(Ak);
          Ak = R*Q;
      end
           M = min(diag(Ak));
end
```

Function 4 - Modified Power Method

```
function [lambda] = power_method(A)
    B = inv(A);

    tol=10^(-16);
    s=size(B);
    n=s(1);
    z=ones(n,1);
    lam=0;

    for k=1:10000
        znew = B*z;
        lam_new = dot(B*znew,znew)/dot(znew,znew);
        if abs(lam_new-lam)<tol
            break
        end
        z = znew/norm(znew,2);
        lam = lam_new;
    end

    lambdainv = lam_new;
    lambda = 1/lambdainv;
end
```

Function 5 - Calculating Percent Error

```
function [err] = errorfunc(approx)
    actual = pi^2;
    err = abs(((approx - actual)/actual)*100);
end
```

MATLAB Code for Section 5

Finding Orthonormal Basis for A_3 (Function 1 and 2)

```
A3 = A(3);
[u1, u2, u3] = gram(A3);
u1 =
    0.8944
   -0.4472
         0
u2 =
    0.3586
    0.7171
   -0.5976
u3 =
    0.2673
    0.5345
    0.8018

dot(u1,u2)
ans =
  -1.6653e-16

dot(u1,u2)
ans =
  -1.6653e-16

dot(u2,u3)
ans =
     0

norm(u1)
ans =
     1

norm(u2)
ans =
    1.0000

norm(u3)
ans =
     1
```

Obtaining QR Factorization for A_3 and A_4 (Given cgs Function)

```
[Q3 R3] = cgs(A(3));
[Q4 R4] = cgs(A(4));
```

```
Q3 =
    0.8944    0.3586    0.2673
   -0.4472    0.7171    0.5345
         0   -0.5976    0.8018


R3 =
   35.7771  -28.6217    7.1554
         0   26.7731  -30.5979
         0         0   17.1047


Q4 =
    0.8944    0.3586    0.1952    0.1826
   -0.4472    0.7171    0.3904    0.3651
         0   -0.5976    0.5855    0.5477
         0         0   -0.6831    0.7303


R4 =
   55.9017  -44.7214   11.1803         0
         0   41.8330  -47.8091   14.9404
         0         0   36.5963  -48.7950
         0         0         0   22.8218
```

Applying QR Algorithm to A_{10}, A_{20} and A_{100}, Finding the Smallest Eigenvalues for Each, and Calculating the Percent Error (Function 3 and 5)

```
QR10 = QR(A(10));
QR20 = QR(A(20));
QR100 = QR(A(100));

QRerror10 = errorfunc(QR10);
QRerror20 = errorfunc(QR20);
QRerror100 = errorfunc(QR100);
```

```
QR10 =
    9.8027
```

```
QR20 =
    9.8512
```

```
QR100 =
    9.8688
```

```
QRerror10 =
    0.6779
```

```
QRerror20 =
    0.1864
```

```
QRerror100 =
    0.0081
```

MATLAB Code for Section 6

Applying Power Method to A_{10}, A_{20} and A_{100}, Finding the Smallest Eigenvalues for Each, and Calculating the Percent Error (Function 4 and 5)

```
power10 = power_method(A(10));
power20 = power_method(A(20));
power100 = power_method(A(100));

Powererror10 = errorfunc(power10);
Powererror20 = errorfunc(power20);
Powererror100 = errorfunc(power100);
```

```
power10 =
    9.8027
```

```
power20 =
    9.8512
```

```
power100 =
    9.8688
```

```
Powererror10 =
    0.6779
```

```
Powererror20 =
    0.1864
```

```
Powererror100 =
    0.0081
```